Benchmarking Reinforcement Learning and Imitation Learning on a Deterministic Platformer: Jump King

William Kwako

Independent Researcher
wkwakow@gmail.com

Abstract

Jump King is a platformer that combines execution precision, hard exploration, and long reward horizons, a combination rarely found together in reinforcement learning (RL) benchmarks. Unlike emulator-based testbeds, we train on the live commercial game via a custom mod that streams state information over TCP, introducing real constraints such as hardware and OS-level timing sensitivity. We compare behavioral cloning (BC), RL, and BC+RL hybrid models across a diverse set of screens, training one agent per screen and chaining them to complete the full game, with a first completion of 9:00.855 (mm:ss), with 256 jumps and 3 falls. BC+RL completes every screen (77.8–100% completion rate), while pure RL succeeds only on the three easiest screens and reaches 0% on every screen with a structural or environmental complication. We additionally find that state representation matters more than hyperparameter tuning, that episode-termination logic acts as an implicit reward signal, and that severely imbalanced action distributions require a class-weighted loss to keep rare actions learnable.

Introduction

Jump King combines several challenges for reinforcement learning (RL) models rarely encountered together: extreme execution precision, hard exploration, and long-reward horizons. For example, a single jump mistimed by a tenth of a second may cause the player to lose a huge amount of progress. Atari57 is a popular testbed that gauges an RL model's competence across many distinct tasks, but relies on environments with direct, low-latency access to game states. Unlike these emulator-based benchmarks, this study uses a live commercial game environment with real-time data extraction. This introduces new constraints that don't exist in simulated environments, such as hardware and OS-level timing sensitivity. This combination of properties makes Jump King a useful setting for directly comparing behavioral cloning (BC), RL, and BC+RL hybrid approaches.

Jump King is a vertical 2D platformer where the goal is to progress upwards by moving left and right (using the arrow keys), and jumping (using the spacebar) (Nexile 2019). The longer the player holds down the spacebar (up to a 0.5833s duration), the farther they jump. The control scheme is simple, but mastering it requires precisely timing how long the spacebar is held across hundreds of jumps. Some jumps have a lenient window of ≈0.20s, while others are extremely precise, at ≈0.05s. When the player jumps beyond their current screen, they progress to the next screen. Some platforms are made of ice, causing the player to slide along them when landing, while other screens contain wind, which affects player movement both on the ground and in the air. In Figure 1, we display a screenshot of screen 3 in Jump King. The player controls The King, located at the bottom of the screenshot.

Figure 1: Screen 3, an early screen in Jump King. The player progresses upward by timing jumps between platforms. Mistiming a jump can drop the player to a previous screen. We analyze BC+RL and pure RL performance on this screen in the Results section.

Prior work on Jump King is limited to informal efforts outside RL research. Code Bullet applied an evolutionary algorithm to a from-scratch JavaScript re-implementation of Jump King (Code Bullet 2022b), completing the game (Code Bullet 2022a). We instead use proximal policy optimization (PPO) — an algorithm designed to improve on historical drawbacks of RL algorithms (Schulman et al. 2017) — in conjunction with BC pretraining. Each agent learned one screen, and RL models were chained together to complete the

full game. 43 agents were trained total; the first completion of the base game was performed in 9:00.855 minutes, with 256 jumps and 3 falls.

Our core findings are as follows:

1. State representation matters more than hyperparameter tuning for Jump King: normalizing the y coordinate (from y to y%360, the height of the screen) tripled BC accuracy on difficult screens.
2. BC materially changes RL's ability to solve a sparse-reward, hard-exploration, precision task: pure RL converges only on the three easiest screens and reaches 0% completion on every screen with a structural or environmental complication, whereas BC+RL completes every screen in the game (77.8–100%), under identical action spaces and rewards.
3. BC data quality strongly gates RL success: with clean demonstrations BC+RL solves even the hardest screens, but on a screen whose human data mixed conflicting routes, pure BC collapsed to 0.6% completion while the same architecture with better-structured data reached 100%.
4. Agent performance can be sensitive to hardware and OS-level timing: one precision-dependent screen required training directly on its deployment hardware, and a single Windows update measurably shifted completion rates on multiple timing-sensitive screens.
5. We introduce learnable-path selection: choosing alternate in-game routes that maximize action/state variety enables the agent to learn screens where human-optimal play is too ambiguous for a memoryless policy.
6. Standard cross-entropy loss fails on screens with severely imbalanced action distributions (like wind screens which have 95%+ no-op frequency). We introduce a class-weighted loss function that inversely weights actions by frequency, which keeps rare actions learnable.
7. Episode termination logic implicitly functions as part of the reward signal: a criterion that seems intuitively correct (terminate episode on height change) can prevent the agent from learning the downstream consequences of its actions, causing the agent to optimize for a local maxima, and failing to discover the global maximum.

Related Work

Code Bullet's approach involved an open-loop evolutionary strategy, in which agents played through the game with predetermined action sequences that could grow in length as the population progressed (Code Bullet 2022b). Each generation retained a single clone of the best performing agent (determined by a fitness function based on height reached), while the rest of the population was produced by selecting parents in proportion to fitness and mutating their action sequences, with an additional mutation targeted at the action where a parent fell. Agents learn through trial and error, but this method does not utilize neural networks, MDPs, or state representation to decide actions in real-time.

Real-time RL research in precision platformers is sparse. We surveyed the precision-platformer genre broadly (Super Meat Boy, Celeste, Getting Over It, I Wanna be the Guy) and found only Celeste had documented attempts, most of which were either informal or unsuccessful. One study attempts to train an RL agent to complete the first level of Celeste using a Python Gymnasium environment, but observes high CPU usage and is unable to finish the first level (boesingerl 2023).

Background

Reinforcement learning (RL) is a subfield of machine learning in which an agent learns to solve a problem through trial and error. At each timestep t, the agent observes a state s_t, selects an action a_t, and receives a reward r_t. The mapping from states to actions is called a policy, denoted π. A sequence of states and actions from an initial state to a terminal state is called an episode. Over many episodes, the agent learns to adjust π to maximize expected reward. A key property of the policies we consider is that they are memoryless. That is, $\pi(a|t)$ only depends on the current state, with no explicit memory of previous states or actions within the episode. Limitations of memoryless policies are discussed further in the Results section.

We use proximal policy optimization (PPO) (Schulman et al. 2017), a general-purpose RL algorithm requiring minimal domain-specific tuning, which is applicable to both discrete and continuous action spaces. Critically, PPO's architecture is compatible with parameter sharing and weight initialization from a separately trained network, enabling BC weight transfer. We initially evaluated a deep Q-network (DQN) (Mnih et al. 2015), but observed the policy converging to a suboptimal solution after several hundred timesteps, far short of the tens of the thousands typically required. This is a known limitation of Q-learning under function approximation (Van Hasselt, Guez, and Silver 2016), where inflated value estimates can cause convergence to suboptimal policies before sufficient exploration has occurred.

PPO differs from historical RL algorithms in three distinct ways. First, it reuses the same rollout batch across multiple epochs of updates, which previous gradient policy methods could not accomplish without policy collapse (Schulman et al. 2017). Second, it achieves comparable stability to TRPO without second-order optimization (Schulman et al. 2015). And third, PPO clips a copy of its samples' probability ratios to prevent it from contributing to that minibatch's gradients past a certain threshold. As rollout data is reused for many minibatch updates, unclipped probability ratios may update the policy into regions that newly collected evidence would disagree with. Therefore, the objective function proposed by PPO is

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(r_t(\theta)\hat{A}_t, \operatorname{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \tag{1}$$

where $r_t(\theta)$ is the ratio between old and new policy probabilities, ϵ is a hyperparameter determining the cutoff range of the copy of r_t, and $\hat{A}_t$ is an estimate of the true advantage (Schulman et al. 2017), computed from sampled rewards and

the learned value function. We use hyperparameters to tune this function further, giving us

$$L^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\Big[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\Big], \tag{2}$$

where L^{VF} is the training loss, and $S[\pi_\theta](s_t)$ is the entropy term. In our experiment, c_1 is the vf_coef hyperparameter, the value function coefficient scalar, and c_2 is ent_coef, a weight on the entropy term that rewards exploration. Specific values used, and their effect on training, are discussed in the Methods/Results section.

Behavioral cloning (BC) is a type of imitation learning whereby agents learn a policy through a set of labeled examples. BC has two main drawbacks. First, compounding error: small mistakes can cause BC to enter states not seen in training data, causing compounding errors to further reduce accuracy (Ross, Gordon, and Bagnell 2011). Second, decision-boundary error: when the correct action changes sharply over a very narrow range (e.g. walking is correct at $x = 252.10$, jumping is correct at $x = 252.25$), the agent can select the incorrect action if data is sparse or states are similar. RL fine-tuning addresses both issues: exploration allows the agent to recover from states outside the training distribution, and training sharpens ambiguous decision boundaries.

A primary drawback of RL in real-world environments is the magnitude of training time required for convergence. Imitation-learning strategies let RL initialize with weights that significantly reduce training time. In one study, Hester et al. (Hester et al. 2018) presented a comparison between Priotized Dueling Double (PDD) DQN and Deep Q-learning from Demonstrations (DQfD), an RL algorithm jump-started with small sets of demonstration data that trains its value function as BC training occurs. PDD DQN never surpassed DQfD on average, and only caught up to DQfD's performance after 83 million steps. Similarly, AlphaStar, an agent trained to play StarCraft II, achieved Grandmaster level (beating 99.8% of players) by using a combination of imitation learning and deep RL neural networks (Vinyals et al. 2019).

Methods

System architecture

We built a custom C# mod that interfaced with Jump King and provided live game data to Python via TCP socket communication. We used existing Jump King documentation for modding, reflection to access metadata, and dnSpy to identify relevant objects. Mod features included reporting game data such as position, velocity, jump duration, and a wind timer; a keylogger that tracked player actions; a tile data scraper that collected platform information; and a teleporter that transported the player to specified coordinates. The C# to Python TCP socket writes data each frame, and the most recent data is pulled from the queue for analysis following action selection.

State representation

We trained a BC agent on a human playing through Jump King, then fine-tuned each model using RL. As a separate agent was trained for every screen, state representation, action spaces, and reward signals varied by screen-specific environmental factors. Most screens started with the following state: `[x, y%360, ceiling, left_wall, right_wall, rel_x_start, rel_x_end]`, where `x` and `y` are the position of the player, `ceiling` is the distance to the ceiling, `left_wall` and `right_wall` are the distances to the nearest walls respectively, and `rel_x_start` and `rel_x_end` are the relative differences to the left and right edge of the platform the player is currently standing on. Distance-based calculations were performed by collecting individual tile data using the custom C# mod, merging them into platforms, then computing Euclidean distances from the player to each surface.

If convergence failed using the above state, screen-specific geometry was analyzed to determine a more appropriate representation. For example, screens with ice platforms, which cause the player to slide on landing, required tracking x velocity, while screens with cyclical wind patterns required adding a timer to track the wind cycle. Furthermore, we observed small discrepancies in tile data on several screens which corrupted state representation. Thus, a simpler, platform-agnostic state representation of `[x, y%360]` was used for several screens.

Our state representation induces POMDP-like ambiguity (the agent cannot fully observe the true game state) even though the underlying game is an MDP at its core. This is for two reasons. First, adding variables to the state increases the time it takes the models to converge, so it's necessary to use the smallest state required for convergence given the live-game environment. And second, the bias-variance tradeoff causes the agent to fail to learn if a state is too general, and fail to generalize if a state is too specific.

Action spaces

Each action in our action space is represented as a tuple describing an input to Jump King, structured like `(left_arrow, right_arrow, spacebar)`, where `left_arrow` and `right_arrow` are durations the right and left arrows are held down, and `spacebar` is the duration the spacebar is held down. Durations vary between 0.05 (1 frame) seconds and 0.5833 (35 frames) seconds, where holding down spacebar for 0.5833s corresponds to the maximum jump height.

Action spaces vary enormously between screens. For simpler screens, like screen 0, only four actions were necessary for completion: (0.2, 0, 0), (0, 0.2, 0), (0.5833, 0, 0.5833), and (0, 0.5833, 0.5833). Or more simply: a short walk in both directions, and a max jump in both directions. For more complicated screens, like screen 17, a combination of small, medium, large, and max jumps were required, totaling to 14 actions. Nearly every screen included four walk actions: 0.1s and 0.2s in both directions. A walk duration of 0.2s was used to cover larger distances, and a walk duration of 0.1s was used for more precise positioning.

Action spaces were selected based on the action frequency distribution according to human playthrough data. In Figure 2, we plot the histogram of human actions on screen 17, with jump duration on the x axis and count on the y axis. The dotted red lines are actions selected for the action space. For this screen, we snapped jump durations to increments of 0.05, then selected the fewest number of actions needed to complete the screen.

Reward signals

Reward magnitudes were tuned empirically over the course of the project as my understanding of the state and action space matured. The scoping column indicates where each term applies under the final scheme. Note that "wind" refers to screens 25-31, and "ice" refers to screens 36-38. Some reward terms differ between the BC+RL and pure RL agents; the scope column notes where. Table 1 below lists the reward scheme:

Type	Description	Value	Scope
Screen completion	On progressing to the next screen	150	non-wind
Screen failure	On falling to a previous screen	-150	non-wind
Height change	Any time height changes	$(\text{new}_y - \text{old}_y)/5$	all
Action cutoff	Actions in one episode exceed the cutoff	-200	all
Speed reward	On completion; higher for fewer actions	$100/n_a$	all
Wind screen completion	On progressing to the next screen	500	wind
Wind screen failure	On falling to a previous screen	-300	wind
Stabilizing jump	Tiny jump opposite to velocity when landing on ice	15	ice
New platform reached	Landing on a new platform (once per episode)	30	BC+RL: ice; RL: all
Proximity to goal	Change in Euclidean distance to goal (first landable space on next screen)	new $-$ old dist.	BC+RL: none; RL: all

Table 1: Reward scheme. “Wind” refers to screens 25–31 and “ice” to 36–38, and n_a is the action count. Some terms differ between BC+RL and pure RL, as noted in Scope.

Which signals were active varied by screen and across the project. Earlier screens were trained under prior iterations, and signals like platform and proximity rewards were applied selectively where a screen needed them.

Episode termination

Episodes were terminated on screen transition, or if the agent hit an action limit. The action limit was set to 100 for non-wind screens, and 250 for wind screens. At episode termination, the agent was teleported back to their training screen. Starting locations for each screen were recorded during human playthroughs, and stored in a dictionary for dynamic access.

BC training and weight transfer

As part of the C# mod described previously, we collected live game data for use in BC training. We transformed the raw data into state-action pairs by categorizing the data by screen, binning the actions using the histrogram-like approach described above, rounding non-selected actions to the nearest selected action, and building state representations for each action.

Our BC NN is a 3-layer MLP, with 256 weights for the input and hidden layers, an 80/20 train/test split, Adam optimization, and 100 epochs of training. We perform a weight transfer step to align the output dimensions of our BC model with the input dimensions of PPO. We do this by copying the two hidden linear layers and final layer into PPO’s action and policy nets.

RL-specific parameters

For most screens, the following RL hyperparameters were used:

Parameter	Description	Value	Reason
n_steps	The number of actions performed before each policy/-value update	2048	A batch size large enough to keep gradient estimates stable. At 0.5-1 actions/second, this meant 1 update approximately every hour
ent_coef	Weights an entropy bonus that discourages the policy from becoming overly deterministic, encouraging exploration.	0.05	We keep this fairly high to encourage exploration
target_kl	A threshold that stops training early if the new policy diverges too much from the old policy	0.02	Empirically adjusted based on agents getting stuck in local maxima
learning_rate	Controls step size during gradient descent	0.0001	Hand-tuned when training plateaued

Table 2: RL hyperparameters used for most screens. Wind screens used ent_coef 0.25 (see text)

Other screens occasionally used different hyperparameters. For example, on wind screens (25-31), an ent_coef of

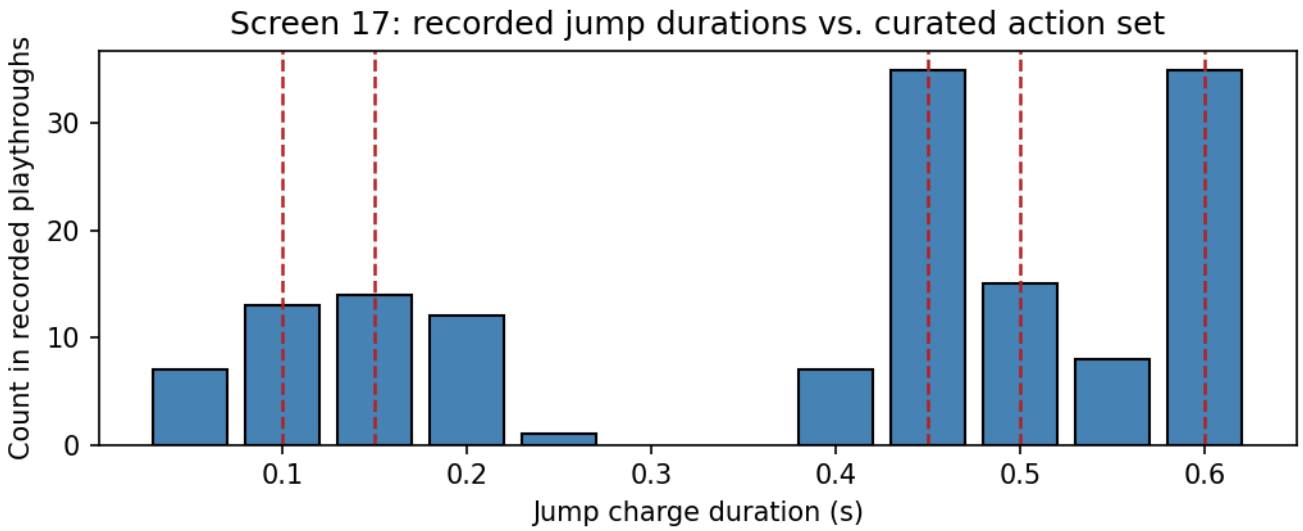

Figure 2: Recorded jump charge durations for Screen 17, compared against the curated action set (dashed lines).

0.25 was used to encourage exploration due to no-op class domination.

Evaluation

To measure performance, we trained each agent for 500 episodes on a given screen and recorded the outcome of each. An episode counts as a success if the agent progresses to the next screen, and a failure if it falls to the previous screen or hits the action timeout (100 actions for non-wind screens, 250 for wind screens). Completion rate is then successes/500.

At evaluation time, actions can be selected in one of two ways. Under deterministic (greedy) evaluation, the agent always takes its single highest-probability action for the current state. Under stochastic evaluation, the agent samples an action from its full probability distribution, so lower-probability actions are still taken occasionally. The evaluation mode is a property of the screen, not the agent: each screen is fixed to one mode, and every model evaluated on that screen (BC+RL, BC, and pure RL) uses the same mode so models can be compared on equal footing.

We use deterministic evaluation by default and set a screen to stochastic in two situations: where the agent converged but the correct action doesn't become the single most-likely one, so greedy selection never takes it; and where greedy selection sends the agent into a repeating loop, for example, on screen 19, where the agent can get stuck jumping into a wall by taking the otherwise-correct action about five pixels off from the correct spot. These screens are run stochastically during full-game playthroughs as well, so evaluating them the same way measures the agent under the exact selection mode it's actually deployed with. Furthermore, because the state representation is partial (the POMDP-like ambiguity noted earlier), two distinct situations can appear near-identical to the agent, so the single highest-probability action is sometimes wrong for the situation it's actually in. Sampling lets the agent escape loops that greedy selection would repeat indefinitely. This is why stochastic evaluation is sometimes necessary in a deterministic game.

Deterministic evaluation is worth understanding in its own right, because it's central to interpreting the pure RL results later. Under greedy selection, a screen is completed only if the correct action is the highest-probability one at every decision point along the path. A single point where a wrong action outranks the right one caps completion there, and because the selection is deterministic, the agent will fail at the same spot on every attempt. This is why a pure RL agent can appear to "almost" solve a screen while still completing it 0% of the time: it plays the same partial solution every run, and gets stuck at the first decision point it never learned.

Results

We choose a sampling of screens across the base game that represent a diverse set of action spaces, reward signals, environmental hazards, difficulty, and overall performance. For both pure BC and pure RL models, the same action space is used to minimize differences across models and provide cleaner comparisons. Pure RL was bounded by design: its wall-clock training cost is immense, ranging from roughly 20 hours for the easier converged screens (screen 0) to over 90 hours for screen 3, which had still not converged when training was stopped at 330k timesteps. However, note that while screen 3's pure RL reward had not plateaued when training stopped, it nonetheless completes the sceen 95.2% of the time, because the action probabilities along the path had already become dominant enough for greedy selection, even as reward continued to climb. Due to the aforementioned wall-clock training issue, we therefore trained pure RL on a smaller subset of screens chosen to span the main axes of task variation: action-space size, hazard type (wind

and ice), precision, and progression direction. This subset is sufficient to support the cross-method comparison while keeping the RL training budget tractable (see Limitations).

Completion rate

In Table 3 below, we display the completion rate percentages compared between BC+RL, BC, and RL models. Completion rate is calculated as the percentage of the time the agent progressed to the next screen (successes/500).

Screen	BC+RL	BC	RL
0	100	76.0	100
1	100	0.6	100
3	100	94.0	95.2
8	98.0	99.0	0.0
10	100	98.6	—
16	94.6	0.0	0.0
17	87.8	0.0	—
26	81.0	29.2	0.0
30	100	100	—
31	100	82.4	—
36	84.4	33.0	0.0
37	77.8	0.0	—
38	84.6	57.6	—
41	92.0	50.2	—

Table 3: Completion rate (%) by screen and method, over 500 evaluation episodes each. "—" marks configurations not evaluated. Pure RL was run on a diverse subset of screens (see Methods). Completion rate is the fraction of episodes in which the agent progressed to the next screen.

BC+RL converged on every screen, with the lowest completion percentage being on screen 37, with a 77.8% completion rate. Pure BC converged on most screens, but struggled on screens with poor data quality, large action spaces, and precise jumps (1, 16, 17, 37). Pure RL was only able to converge on easy screens (0, 1, 3). It was not able to complete any of the other screens it was evaluated on. We examine one such failure, screen 36, in the discussion section.

Reward and episode length comparison between models

Next, we compare episode reward and length between BC+RL models and RL models trained on selected screens. Below, we display a subplot comparing average reward per episode and average number of steps per episode against total timesteps, plotted for BC+RL and RL models on screen 3.

The differences here are stark. The BC+RL model converges after 8k time steps, and the pure RL model continues to rise after 300k time steps. Average episode length never increases for BC+RL, and RL sees a spike in actions, then a precipitous drop and a leveling off. This graph starts low as episodes end almost immediately with RL falling to a previous screen. At 110k timesteps, episode length stabilizes while average reward continues to increase.

BC+RL screen comparison

We also examine how screen difficulty shapes BC+RL learning. Difficulty was determined by action space diversity, required jump precision, and, more informally, how difficult each screen was for the player across playthroughs. The clearest case is screen 37, the hardest screen in the game: it has an action space size of 10, icy platforms, mostly non-maximum jumps, and the trickiest jump in the game, which requires jumping while the player is still sliding from a previous jump. We plot its average episode reward against timesteps in Figure 4 below.

Screen 37's average reward per episode starts the lowest of any screen we trained, at nearly zero, but rises the fastest, as RL fine-tunes a starting policy that BC alone leaves far short of a solution. This contrasts with easier screens, where BC already supplies a near-optimal policy and RL has little left to do, Screen 10, for example, starts high near 185 and climbs only to roughly 217, with its episode length falling slightly from about 11.4 actions to 10.6 as training proceeds. The harder the screen, the more of the final policy comes from RL rather than BC.

BC+RL seed robustness

To test whether BC+RL's success on a given screen depends on a lucky seed, we trained five agents on screen 16, each with a different random seed. Screen 16 was chosen because it demands above-average precision, as its jump window is among the tighter ones in the game, but the screen overall isn't as punishing as screen 37. This makes it a meaningful stress test without being an outlier. We used a mean-episode-reward threshold of 160 as the convergence bar, since the full game was completed with a screen 16 agent whose convergence reward was 160. Reaching that bar is what we treat as "playthrough-ready."

We plot raw mean episode reward against timesteps for all five seeds below, with the deployment threshold marked.

All five seeds follow the same learning trajectory and cross the deployment threshold, reaching it at 36045 ± 12424 timesteps (n=5). Seed 16a was trained substantially longer (to 61k timesteps) because it served as the agent in the full-game playthrough, while seeds 16b–16e were stopped once they crossed the threshold, which is why their curves terminate earlier. The seeds converge to somewhat different absolute reward values, but all clear the bar that determines playthrough readiness, indicating that BC+RL's ability to solve this precision-critical screen is robust to seed choice rather than an artifact of a single fortunate run.

Discussion

Findings from data

BC+RL wins broadly, outperforming nearly every BC-only or RL-only model. The notable exceptions are screens 0, 1, and 3, where BC+RL achieves slightly higher completion times and action counts than the RL-only models, and screen 8, where BC+RL is marginally worse than BC alone. In all four cases, however, the differences fall within one standard deviation. The two single-method baselines fail in characteristic and opposite ways. BC-only models perform well overall

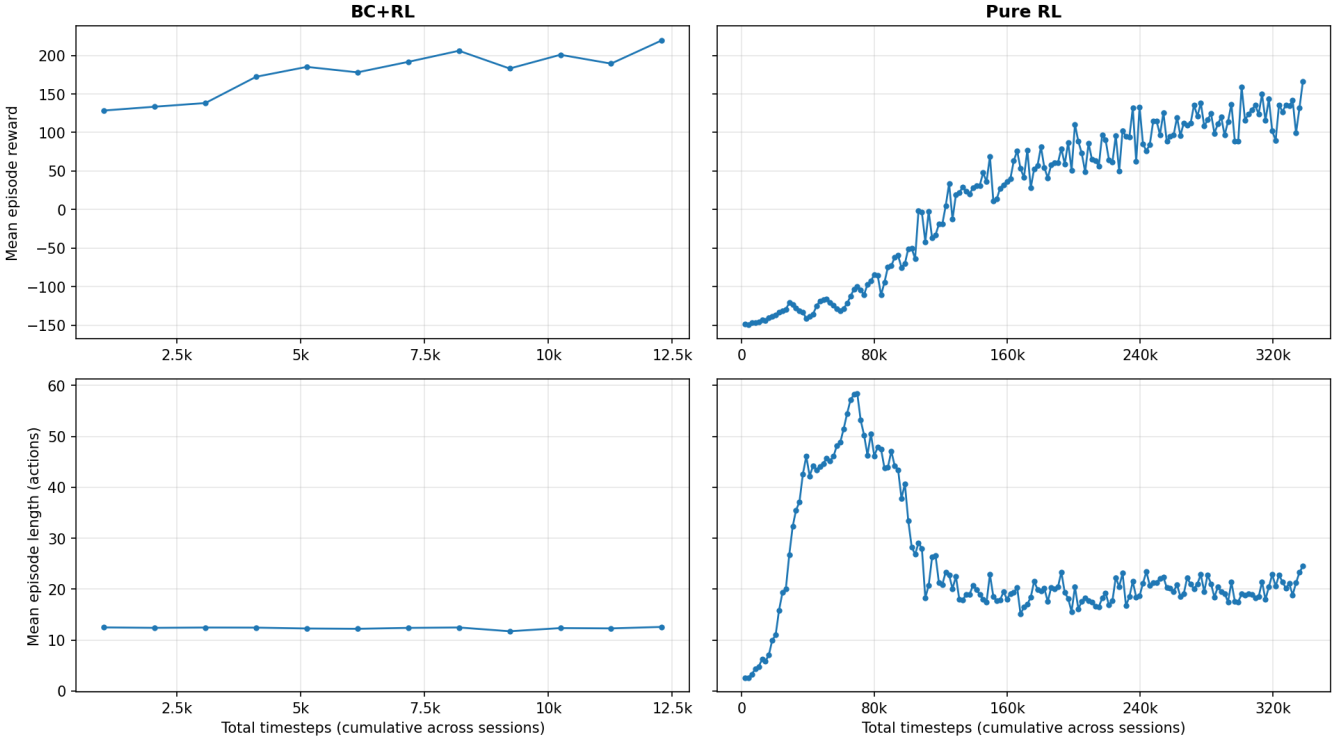

Figure 3: BC+RL converges at 8K time steps, while pure RL has not converged after 335K. Episode length is static for BC+RL, whereas pure RL requires 110K time steps to stabilize.

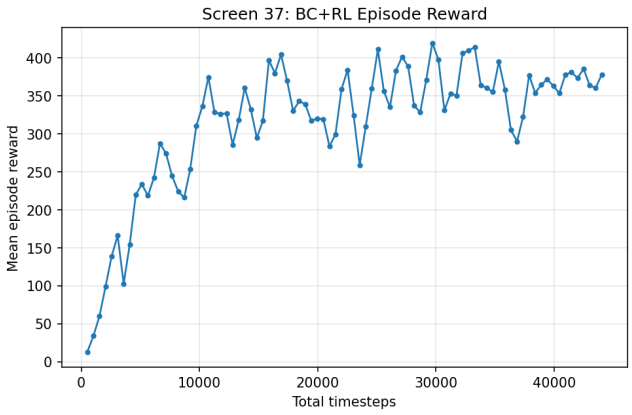

Figure 4: BC reward starts near zero, but RL rapidly increases reward. Note that screen 37 was trained using n_steps=512 instead of the typical 2048, which causes noisier updates.

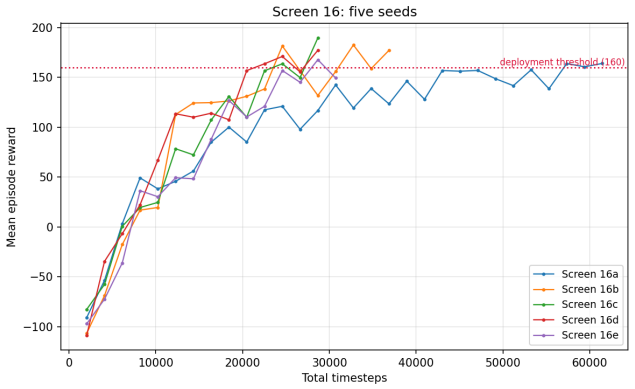

Figure 5: All five seeds on screen 16 cross the deployment threshold of 160.

but get stuck more often than either alternative. On screen 0, pure BC took 65.30 ± 65.35 actions (see Appendix, Table 5) to complete a screen where falling is impossible, and informal observation showed the agent walking back and forth, repeating actions that were frequent in the human playthroughs but that do not advance the screen — a textbook failure mode of imitation learning. BC also fails catastrophically when its data is bad. On screen 1, human playthrough data includes multiple routes through the screen which conflict with each other. BC+RL and RL both achieve a 100% complete rate, while pure BC sits at only 0.6%, taking 722.0 ± 858.61 seconds (see Appendix, Table 4) and 921.6 ± 1079.3 actions on the rare occasions it succeeds. BC occasionally stumbles into a solution after roughly 920 actions, but success is an

exception.

Pure RL fails in the opposite direction, unable to learn many screens (8, 16, 26, 36) under the combined pressure of sparse rewards, long time horizons, and the rarity of selecting the correct action, only converging on the three easiest screens (0, 1, 3). Every screen with a structural or environmental complication, such as downward progression (8), sub-0.1s precision (16), wind (26), or ice (36), defeated it, even though action spaces and rewards were identical to BC+RL. Screen 36 shows the failure explicitly. Mean reward rose from roughly -114 to -60 over about 80k timesteps before plateauing, and the agent even discovered the correct technique to land on ice platforms without falling (influenced by a corresponding reward signal), yet learning stalled short of a solution. Under greedy evaluation the agent reached the second-to-last platform in all 500 episodes and then failed the final jump in all 500. The terminal jump never became the highest-probability action, so the agent played the level perfectly up to that point and then failed at the same spot every run. The terminal states were reached too rarely during training for their reward to consolidate. This is precisely the failure BC+RL repairs, since demonstration initialization places the agent at the frontier densely enough to reinforce those final jumps, clearing the screen 84.4% of the time.

Screen difficulty leaves a clear imprint on completion rates, driven by state representation and environmental hazards. Early screens tend toward higher completion rates and lower variance simply because they are easier, an advantage that data quality, hyperparameter tuning, and state representation cannot fully erase elsewhere. Difficulty also shapes the learning curves: easier screens such as screen 10 show more gradual learning because average reward already begins high and near-optimal, leaving little room to improve. Reward and episode length, meanwhile, do not always move together. On screen 3, average episode length rises, peaks near 58 actions, then drops sharply before stabilizing around 20. Episodes are short early on because the agent falls quickly. As it learns to avoid falling, they lengthen, and informally, we observed the agent stalling to stay alive rather than progressing — jumping in place rather than advancing — which inflates length without improving completion. Length then falls as the agent begins completing the screen successfully, since a completion is shorter than prolonged stalling. We did not log per-episode termination causes during training, so we cannot quantify how much of the peak reflects episodes hitting the action cutoff versus long but sub-cutoff stalling.

Finally, the head-start BC provides is visible from the very first timesteps. On screen 1, BC+RL begins at roughly -20 average reward against pure RL's -100, and on screen 3 it begins near 130, (close to its eventual convergence value of 250) while pure RL starts at -150.

Core findings

The central result is that BC materially changes RL's ability to solve a sparse-reward, hard-exploration, precision task. Pure RL achieves a 0% completion rate on many screens, yet no screen proved uncompletable with BC+RL. The lowest completion rate we recorded for any BC+RL agent is 77.8%, on screen 37. BC supplies a near-optimal path almost immediately, where pure RL must discover both reward and action sequencing from scratch, and the hybrid combines BC's head-start and near-correct sequencing with RL's capacity to refine behavior and to adapt to situations the human demonstrations never covered.

Three further findings emerged, each with a clear resolution. First, agent performance is sensitive to hardware and OS-level timing. One precision-dependent screen (37) required training directly on its deployment hardware, and a single Windows update coincided with a measurable shift in completion rates on several timing-sensitive screens. We recovered some of this by restarting machines before training sessions and raising Jump King's process priority in Task Manager. Second, episode termination logic implicitly functions as part of the reward signal. A criterion that seems intuitively correct — terminating an episode on any height change — can prevent the agent from learning the downstream consequences of its actions, driving it toward local maxima rather than the global one. Early models terminated on height gain, which caused the agent to often get stuck near walls, unable to execute the action sequences that non-vertical progression requires. Terminating instead on screen success or failure resolved this. Third, standard cross-entropy loss fails on screens with severely imbalanced action distributions. On wind screens, no-ops constitute roughly 95% of recorded actions, so a standard loss causes them to dominate learning even though jumps are essential to completion. We used a class-weighted loss that weights actions inversely to their frequency keeps the rare actions learnable.

Design considerations

Several decisions were open design choices, where we selected one of multiple defensible paths. The first is that state representation often matters more than hyperparameter tuning for Jump King. Normalizing the y-coordinate to y % 360 (the height of a single screen) tripled BC accuracy on difficult screens, raising screen 36 from roughly 0.25 to 0.8. A second choice was to train per-screen specialists rather than one general agent, which struggled to learn the full game for several reasons: class balance varies so widely across screens that BC could not weight actions appropriately for all of them at once; states across different screens appeared too similar to the agent, producing incorrect action selection; and screens higher in the game were rarely reached during training and saw little training time.

A third choice concerns the action space, which we curated for each screen from the action-frequency distribution in human playthroughs, choosing the smallest set that still covered the screen. A small action space speeds convergence and removes rarely-useful choices, but it is only as complete as the demonstrations provided, as actions not seen in human playthrough data are unavailable to the agent. We applied this same curated space to the pure-RL and BC-only agents so that cross-model comparisons reflect the learning method rather than differences in available actions. A uniform action grid was possible but would have slowed training substantially and lowered comparison value. Reward configurations, by contrast, were iterated manually rather than parameterized, with consequences for reproducibility that we

discuss under Limitations. Finally, we deliberately selected alternate in-game routes for BC data collection that maximized state variety, which let the agent learn screens where human-optimal play is too ambiguous for a memoryless policy. A longer route with varied platform heights yields more distinctive states than one crossing several platforms at the same height, which was particularly useful for completing wind screens.

Limitations

Next, we discuss three limitations of the study: run-level reproducibility, wall-clock training time, and value function fit.

Run-level reproducibility

Reward shaping was iterated rapidly and often by hand over the course of the project, with terms adjusted or toggled per-screen as different approaches were tested. As a result, the exact reward configuration for a specific historical training run is not always reconstructable from the current code. This is a limitation of per-run reproducibility, not of the reported results, which reflect the final trained agents' saved weights. Parameterizing every reward term across every refactor would have added hours per iteration on configurations that were usually discarded, so rapid manual iteration was the more practical choice. The Methods reward table reflects the final scheme, with earlier iterations noted there.

Wall-clock training time

One of the primary downsides of training reinforcement learning agents in real-time environments is the amount of wall-clock time it takes for agents to converge. BC+RL agent training time varies wildly from screen to screen, with easier screens converging in several thousand time steps and others needing tens of thousands. RL agent training time is even longer: at a minimum, each screen that converged at all required at least 100,000 timesteps, and the hardest screens did not converge within any practical budget. For example, screen 3 trained for 90.5 hours, and had yet to converge. A small adjustment to one model in the middle of training requires re-training that screen, delaying convergence by dozens of hours. This is the main reason why so few RL screens were trained until convergence. Additionally, it isn't feasible to re-train all previously converged screens when a better method is later found, and it's crucial to carefully verify adjustments to the code or hyperparameters, as mistakes bear a high time cost.

Value function fit

On many screens the value loss did not meaningfully decrease over training, and explained variance remained near zero, meaning the critic never learned to predict returns accurately. This was not simply a matter of under-training the critic, as freezing the policy and training the value function alone across several rollouts did not meaningfully improve the fit. We also tried initializing the value function to predict normalized height, on the intuition that higher positions are worth more. This did not help, and on screens requiring lateral or downward progress it was misleading, because reward in Jump King comes from transitioning to a higher platform, not from height itself. Identifying which positions afford such transitions requires tile-level geometric analysis or simulation that the live-game setup does not support. PPO still improved the policy in all cases since its advantage estimates were sufficient to guide learning, so the value function should not be read as an accurate return model. The most likely explanation is the sparsity and high variance of the reward signal combined with a state that omits the geometric information needed to predict value. However, we did not run the experiments needed to fully separate these.

Conclusion

We created a reinforcement learning agent to play and complete Jump King, from creating a full pipeline directly interfacing with the live game via C# memory extraction and TCP socket communication, to a custom Gymnasium environment. Agents can reliably complete all 43 screens, and the first full game completion was achieved in 9:00.855 minutes, with 256 jumps and 3 falls. BC and RL are complementary: BC provides a near-correct starting policy almost for free, and RL then refines it and handles what human demonstrations miss. The hybrid model was able to learn every screen in the game, with completino metrics rangeing from 77.8% to 100%. For Jump King, state representation matters more than hyperparameters, as evidenced by a single state change (y to y%360), which tripled BC accuracy. Jump King was a useful controlled test bed where the same task can be completed by several different models and compared with minimal model differences.

Appendix

Additional per-screen metrics

The following tables report secondary per-screen metrics under the same evaluation protocol described in the methods section (500 episodes per configuration, time and action counts reset only on success). Table 4 reports completion time, Table 5 reports actions-to-completion, and Table 6 reports timesteps trained.

Screen	BC+RL	BC	RL
0	5.0 ± 0.0	29.2 ± 26.4	4.6 ± 0.0
1	7.2 ± 0.8	921.6 ± 1079.3	6.8 ± 0.8
3	8.4 ± 0.8	9.3 ± 3.2	9.2 ± 3.6
8	10.0 ± 1.7	9.9 ± 1.1	*no successes*
10	8.7 ± 0.3	9.0 ± 0.3	—
16	15.0 ± 1.6	*no succ.*	*no successes*
17	13.5 ± 2.8	*no succ.*	—
26	31.0 ± 14.6	119.9 ± 119.2	*no successes*
30	24.8 ± 0.4	24.9 ± 0.1	—
31	11.8 ± 1.4	14.3 ± 5.4	—
36	11.5 ± 7.0	38.6 ± 30.5	*no successes*
37	10.6 ± 3.9	*no succ.*	—
38	9.8 ± 4.0	11.7 ± 6.9	—
41	7.5 ± 1.5	10.4 ± 7.8	—

Table 4: Mean completion time in seconds ($\pm$ SD) by screen and method, over successful episodes only. "*no successes.*" marks configurations evaluated but never successful; "—" marks configurations not evaluated. Times are reported to one decimal place due to hardware and OS-level timing sensitivity (see Methods).

Screen	BC+RL	BC	RL
0	5.00 ± 0.00	65.30 ± 65.35	4.00 ± 0.00
1	6.01 ± 1.28	722.0 ± 858.61	5.34 ± 0.57
3	12.27 ± 1.10	13.95 ± 4.34	11.35 ± 5.48
8	10.43 ± 1.48	10.38 ± 1.05	*no successes*
10	9.92 ± 0.83	11.80 ± 0.89	—
16	14.64 ± 1.40	*no succ.*	*no successes*
17	16.72 ± 2.16	*no succ.*	—
26	94.89 ± 36.57	367.14 ± 362.16	*no successes*
30	83.83 ± 1.92	84.00 ± 0.81	—
31	23.72 ± 2.97	30.46 ± 14.18	—
36	10.98 ± 2.90	23.07 ± 16.24	*no successes*
37	12.63 ± 3.54	*no succ.*	—
38	10.50 ± 4.79	11.02 ± 6.07	—
41	7.51 ± 1.30	8.52 ± 3.97	—

Table 5: Mean number of actions to completion ($\pm$ SD) by screen and method, over successful episodes only. "*no successes.*" marks configurations evaluated but never successful; "—" marks configurations not evaluated.

Screen	BC+RL	RL
0	10240	104448
1	43008	118784
3	8192	337920
8	26624	*did not converge*
10	6144	—
16	57344	*did not converge*
17	24576	—
26	77824	*did not converge*
30	10240	—
31	8192	—
36	23552	*did not converge*
37	16896	—
38	38912	—
41	1792	—

Table 6: Timesteps each model was trained, by screen. For BC+RL, training was stopped once mean episode reward plateaued. For pure RL, "*did not converge*" marks runs stopped before reward plateaued; "—" marks configurations not evaluated.

References

boesingerl. 2023. celesteRL. GitHub repository. Accessed: 2026-08-03.

Code Bullet. 2022a. AI Learns to Play JUMP KING. YouTube. Accessed: 2026-07-22.

Code Bullet. 2022b. Jump-King. GitHub repository. Accessed: 2026-07-22.

Hester, T.; Vecerik, M.; Pietquin, O.; Lanctot, M.; Schaul, T.; Piot, B.; Horgan, D.; Quan, J.; Sendonaris, A.; Osband, I.; Dulac-Arnold, G.; Agapiou, J.; Leibo, J.; and Gruslys, A. 2018. Deep Q-learning from demonstrations. *Proc. Conf. AAAI Artif. Intell.*, 32(1).

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.

Nexile. 2019. Jump King. Microsoft Windows.

Ross, S.; Gordon, G. J.; and Bagnell, J. A. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. arXiv:1011.0686.

Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.; and Moritz, P. 2015. Trust Region Policy Optimization. In Bach, F.; and Blei, D., eds., *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, 1889–1897. Lille, France: PMLR.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with Double Q-learning. *Proc. Conf. AAAI Artif. Intell.*, 30(1).

Vinyals, O.; Babuschkin, I.; Czarnecki, W. M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D. H.; Powell, R.; Ewalds, T.; Georgiev, P.; Oh, J.; Horgan, D.; Kroiss, M.; Danihelka, I.; Huang, A.; Sifre, L.; Cai, T.; Agapiou, J. P.; Jaderberg, M.; Vezhnevets, A. S.; Leblond, R.; Pohlen, T.; Dalibard, V.; Budden, D.; Sulsky, Y.; Molloy, J.; Paine, T. L.; Gulcehre, C.; Wang, Z.; Pfaff, T.; Wu, Y.; Ring, R.; Yogatama, D.; Wünsch, D.; McKinney, K.; Smith, O.; Schaul, T.; Lillicrap, T.; Kavukcuoglu, K.; Hassabis, D.; Apps, C.; and Silver, D. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782): 350–354.